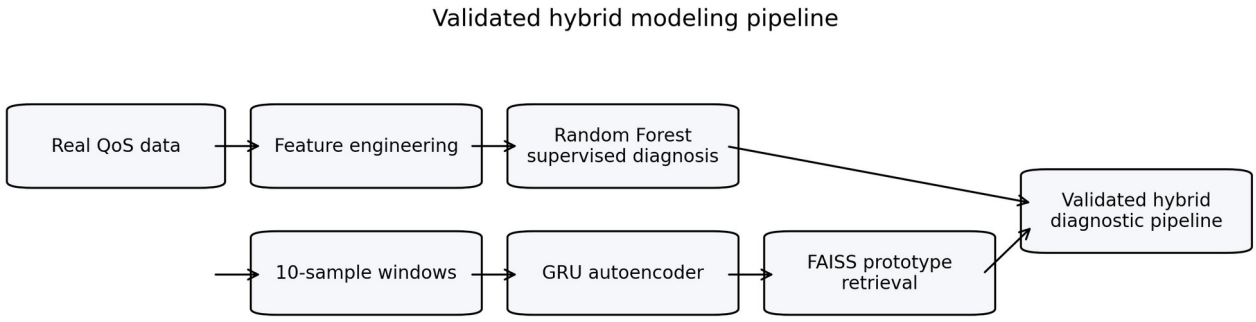
# QoS Buddy Diagnostic Agent Modeling Validation Report

*Revised, explanatory version for modeling validation*

| | |
|---|---|
| **Benchmark scope** | **Real-data benchmark restricted to RC_CAPACITY_OVERLOAD and RC_TRANSPORT_DELAY for supervised training** |
| **Validated winners** | Supervised: Random Forest \| Temporal: GRU \| Prototype: FAISS |
| **Temporal setting** | Window size = 10 samples; best temporal model uses GRU latent embeddings |
| **Purpose of this report** | Explain the training columns, feature engineering, model choices, architectures, evaluation strategy, and presentation-ready results |

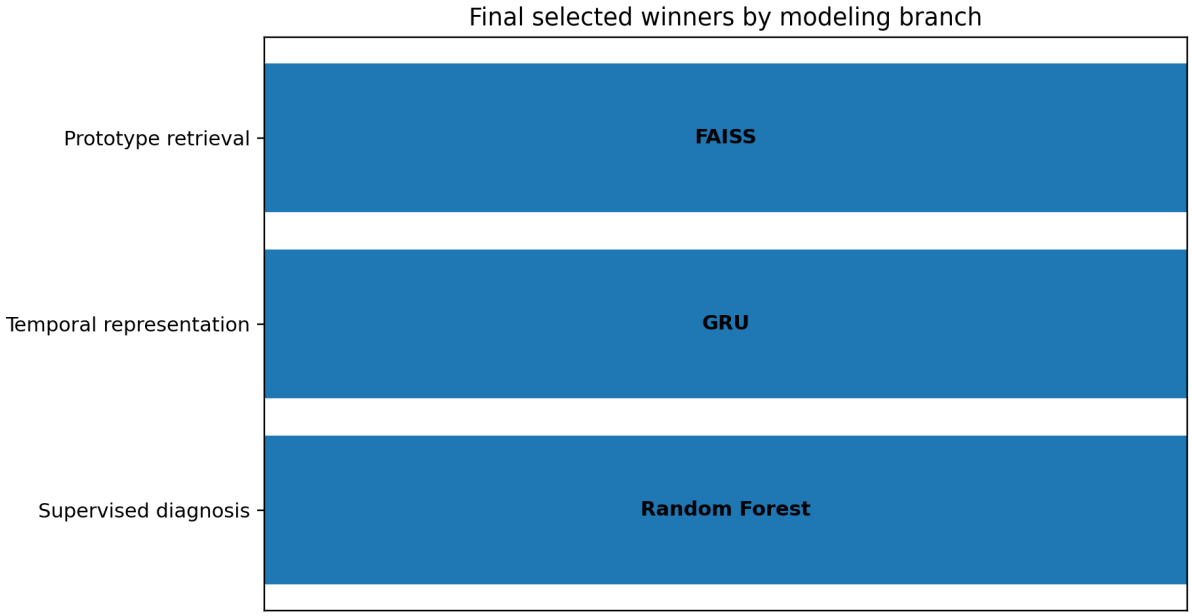



*Validated hybrid modeling pipeline used in this benchmark.*

## Executive Summary

This revised report presents a clearer and more explainable summary of the modeling validation carried out for the QoS Buddy Diagnostic Agent. The benchmark was intentionally constrained to the two root causes that were supported robustly by real-data weak supervision: RC_CAPACITY_OVERLOAD and RC_TRANSPORT_DELAY. Rare radio-side evidence such as RC_CQI_MISMATCH was kept exploratory and not forced into supervised training.

- The corrected supervised benchmark selected Random Forest as the best baseline after removing 14 risky shortcut features and retaining 104 training columns.
- The temporal sequence branch selected GRU because it achieved lower validation reconstruction loss (0.391 vs 0.402) and lower mean test reconstruction error (0.455 vs 0.508).
- The prototype branch selected FAISS because it matched NearestNeighbors on validation quality but reduced validation search time from 0.130s to 0.001s.
- The validated outcome is therefore a coherent hybrid pipeline: Random Forest for supervised diagnosis, GRU Autoencoder for temporal representation, and FAISS for prototype retrieval support.




*Final winners selected for each modeling branch.*

## 1. Alignment with Diagnostic Agent Business Objectives

The Diagnostic Agent is not only a modeling exercise; it exists to support business objectives that matter in network operations. The chosen models were therefore evaluated not only on raw performance but also on how well they fit the operational role of the agent.

| Business objective | Modeling response | Why it matters |
|---|---|---|
| Fast first diagnosis | Random Forest on engineered KPIs | Provides an immediately usable baseline for operational triage on structured snapshot features. |

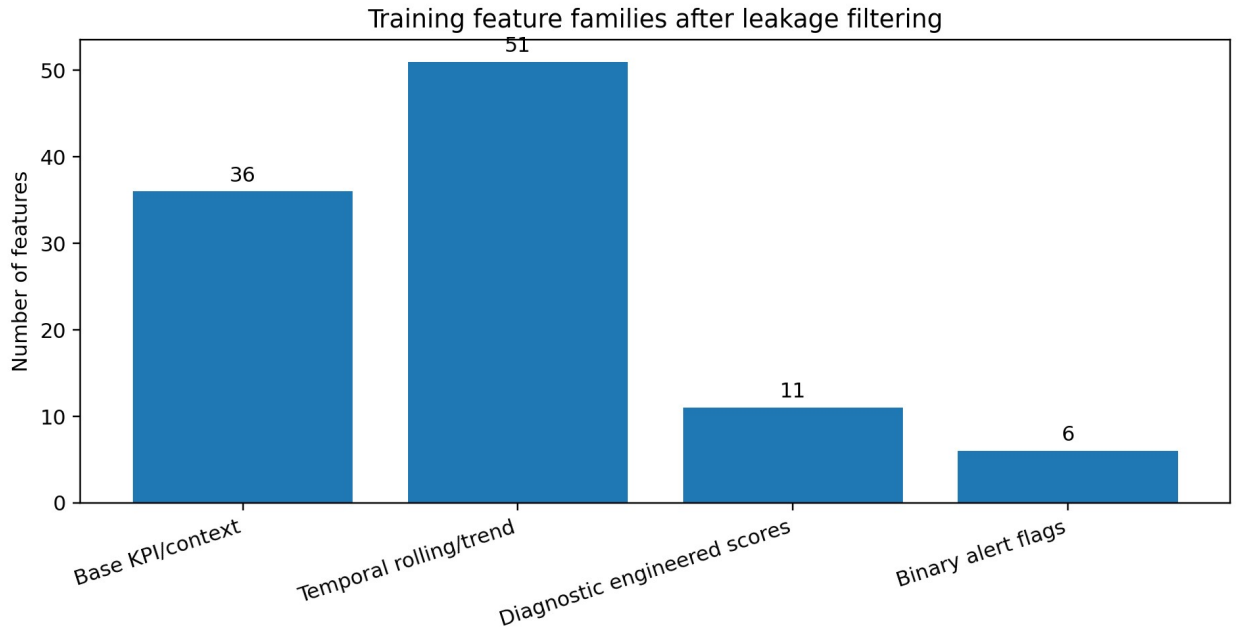
| | | |
|---|---|---|
| Temporal understanding of evolving QoS problems | GRU Autoencoder on 10-sample windows | Captures how degradation evolves across time instead of relying only on one row. |
| Explainable retrieval of similar past cases | FAISS on GRU latent embeddings | Supports a case-based reasoning layer and makes the agent easier to explain to operators. |
| Reliable decision making | Real data only + chronological split + purge gap | Reduces unrealistic optimism and keeps the benchmark aligned with deployment reality. |

## 2. Training Columns and Data Engineering

The final corrected supervised notebook trained on 104 columns. These columns came from the benchmark feature matrix after removing 14 risky or potentially leaky fields such as anomaly_flag, anomaly_score, baseline_phase, traffic_confidence, skip_for_training, and other process/completeness shortcuts.

Instead of treating the feature set as a flat list, it is clearer to understand it as a set of feature families. The benchmark combines raw KPIs, engineered diagnostic signals, and rolling temporal summaries produced during Notebook 3.

| Feature family | Count | Examples |
|---|---|---|
| Core QoS KPIs | 7 | cssr_proxy_pct, jitter_ms, latency_ms, mos_estimate, packet_loss_pct, tcp_retransmit_rate ... |
| Radio / RAN KPIs | 26 | bler_delta, bler_mcs_stress, bler_pct_model, bler_pct_model_delta_3, bler_pct_model_delta_5, bler_pct_model_rollmean_3 ... |
| Congestion / capacity / router | 20 | active_connections, bandwidth_util_pct, bandwidth_util_pct_delta_3, bandwidth_util_pct_delta_5, bandwidth_util_pct_rollmean_3, bandwidth_util_pct_rollmean_5 ... |
| Engineered diagnostic features | 14 | congestion_index, flag_high_util, flag_latency_crit, flag_latency_warn, flag_pl_crit, flag_pl_warn ... |
| Rolling / delta temporal features | 33 | jitter_ms_delta_3, jitter_ms_delta_5, jitter_ms_rollmean_3, jitter_ms_rollmean_5, jitter_ms_rollstd_3, jitter_ms_rollstd_5 ... |
| Handover / event support | 4 | handover_count, handover_event, ho_success_rate_pct, jitter_increasing |




*Feature families used by the corrected benchmark.*

### 2.1 Did we do data engineering?

Yes. The benchmark is not trained on raw columns only. The pipeline explicitly creates diagnostic engineered features and temporal summaries so that the models see network behavior in a more meaningful way.

- BLER normalization into a stable modeling column (bler_pct_model).
- Diagnostic engineered features such as bler_pressure_score, bler_sinr_gap, bler_mcs_stress, handover_instability_index, transport_pressure_score, throughput_loss_explainer, congestion_index, radio_efficiency_score, and radio_vs_transport_score.
- Operational ratios and flags such as latency_jitter_ratio, throughput_util_ratio, flag_latency_warn, flag_pl_warn, flag_high_util, and flag_queue_high.
- Rolling and delta temporal summaries over recent samples, including rollmean, rollstd, and delta features for latency, jitter, packet loss, throughput, BLER, queue length, and bandwidth utilization.
- Sequence windows of 5, 10, and 20 samples, with 10-sample windows chosen as the main temporal benchmark.

## 3. Models Chosen, Why They Were Chosen, and How They Fit the Agent

The model set was chosen to reflect the hybrid design of the Diagnostic Agent. Rather than forcing one model to do everything, the benchmark validates three complementary roles: a tabular diagnostic classifier, a temporal representation model, and a prototype-retrieval backend.

| Branch | Candidates | Winner | Why this family fits the agent |
|---|---|---|---|
| Supervised tabular diagnosis | Random Forest, XGBoost | Random Forest | Tree ensembles are strong on structured KPI tables, handle non-linear |

| | | | |
|---|---|---|---|
| | | | interactions well, and are easy to benchmark quickly. |
| Temporal sequence representation | LSTM AE, GRU AE | GRU | Recurrent autoencoders align naturally with fixed-length QoS windows and can learn sequence representations without requiring synthetic data. |
| Prototype retrieval | NearestNeighbors, FAISS | FAISS | Prototype retrieval complements supervised diagnosis by surfacing similar historical patterns in latent space. |

# 4. Model Explanations and Architecture

## 4.1 Random Forest

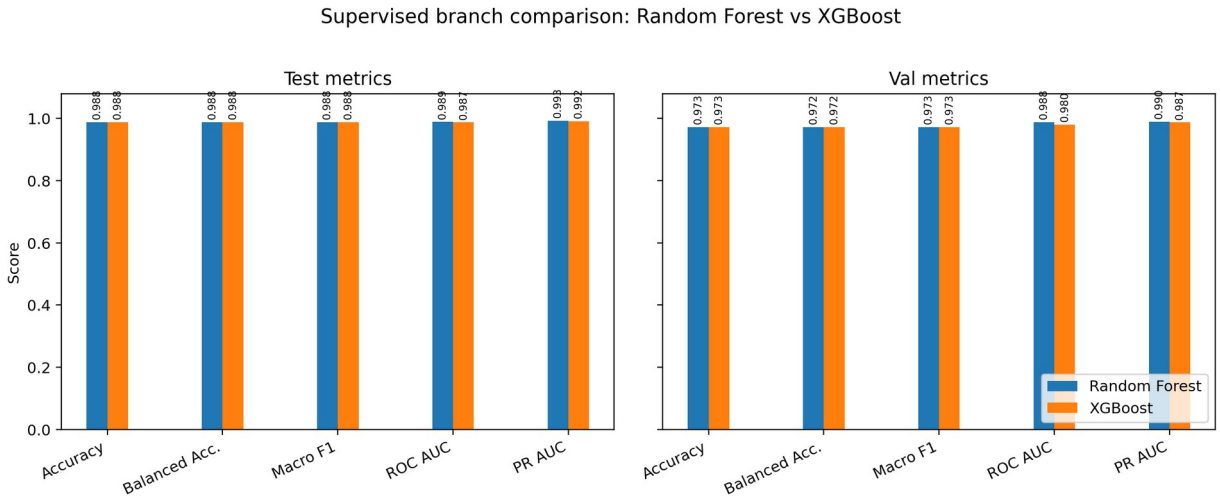
Random Forest is an ensemble of decision trees trained in parallel. Each tree is trained on a bootstrapped subset of the training rows and a random subset of the available features. At inference time, each tree votes, and the forest aggregates the votes to produce the final class prediction.

- Strength in this benchmark: robust baseline for tabular engineered KPIs.
- Why it integrates well into the agent: it can provide a fast first diagnostic guess from the current feature snapshot.
- Operational benefit: easy feature-importance interpretation and low training time.

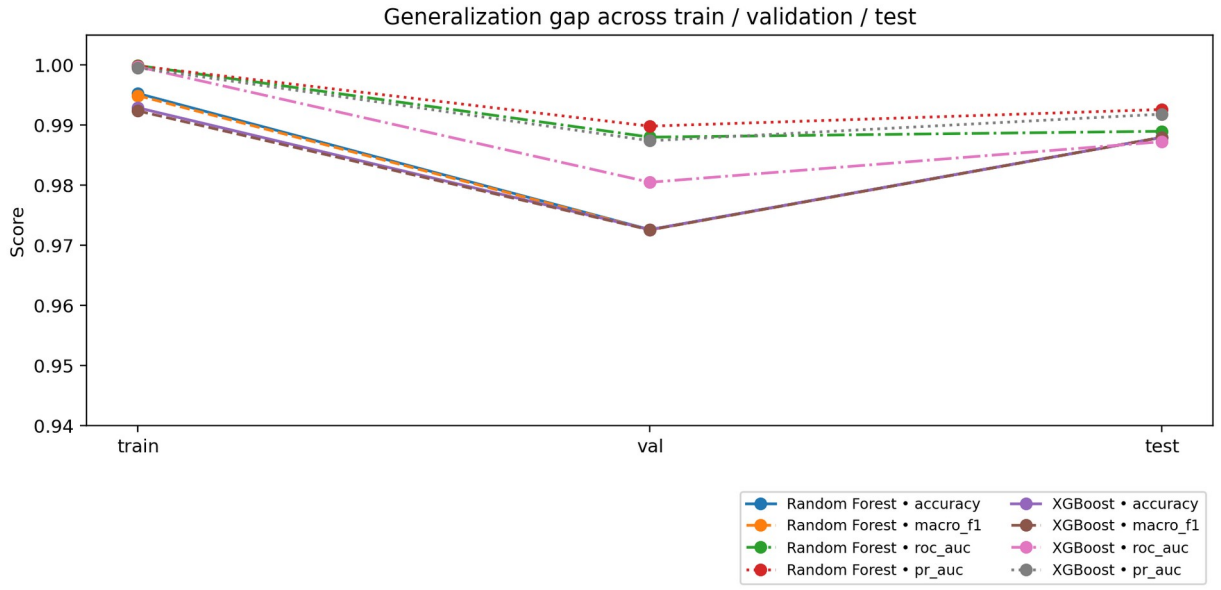
## 4.2 XGBoost

XGBoost is a gradient-boosted tree ensemble. Unlike Random Forest, it builds trees sequentially. Each new tree focuses on correcting the errors left by the previous trees. This often gives excellent performance on structured tabular data, especially when feature interactions are complex.

- Strength in this benchmark: strong structured-data benchmark comparator.
- Why it integrates well into the agent: it is a natural candidate when diagnostic KPIs are heavily engineered and non-linear.
- Operational trade-off: often powerful, but less straightforward than Random Forest when the benchmark is small and already easy to separate.




*Corrected supervised comparison between Random Forest and XGBoost.*




*Train/validation/test generalization-gap view for the corrected supervised benchmark.*

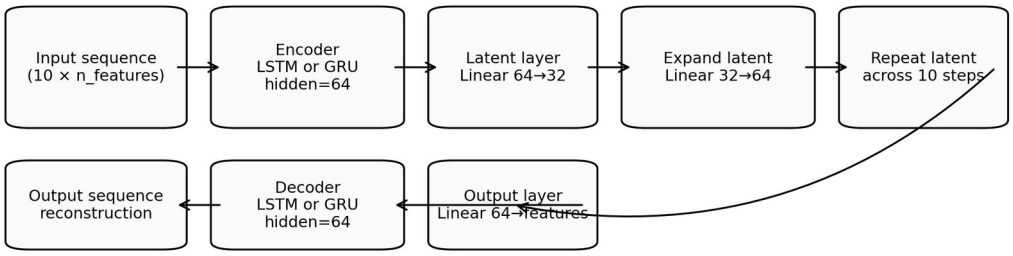
## 4.3 Autoencoders for the Temporal Branch

The temporal branch uses recurrent autoencoders. An autoencoder learns to compress a sequence into a compact latent representation and then reconstruct the original sequence from that representation. In this benchmark, the sequence is a 10-sample QoS window.

| Stage | Layer / operation | Role in the model |
|---|---|---|
| Input | (10, n_features) | Receives the ordered QoS window. |
| Encoder recurrent layer | LSTM or GRU, hidden_dim = 64 | Compresses the sequence dynamics into a hidden state. |
| Latent projection | Linear(64 → 32) | Builds a compact latent vector that summarizes the window. |
| Latent expansion | Linear(32 → 64) | Transforms the latent vector back into decoder space. |
| Repeated latent sequence | Repeat across 10 steps | Provides the decoder with a full-length latent sequence seed. |

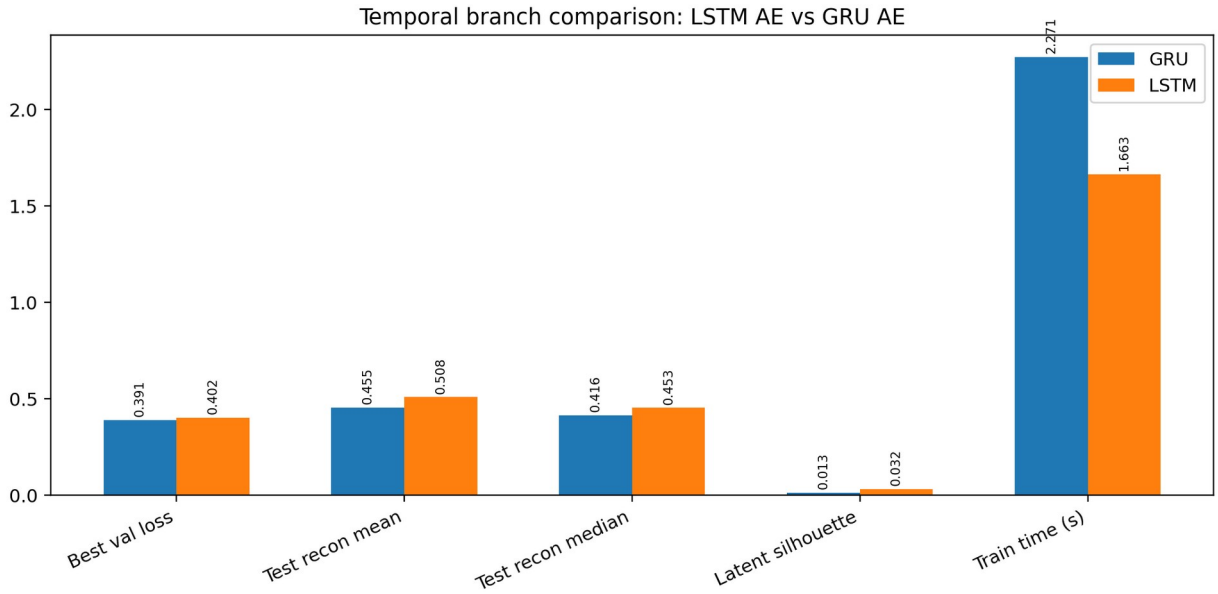
| | | |
|---|---|---|
| Decoder recurrent layer | LSTM or GRU, hidden_dim = 64 | Reconstructs the temporal dynamics from latent space. |
| Output projection | Linear(64 → n_features) | Maps decoder states back to the original feature dimension. |

Autoencoder architecture used in the temporal branch




*Simplified architecture of the recurrent autoencoder branch.*

LSTM and GRU were compared because both are standard recurrent models for sequential data. GRU is slightly simpler and often trains faster; LSTM can sometimes represent longer dependencies more richly. The benchmark was used to determine which one reconstructs QoS windows better under the current settings.

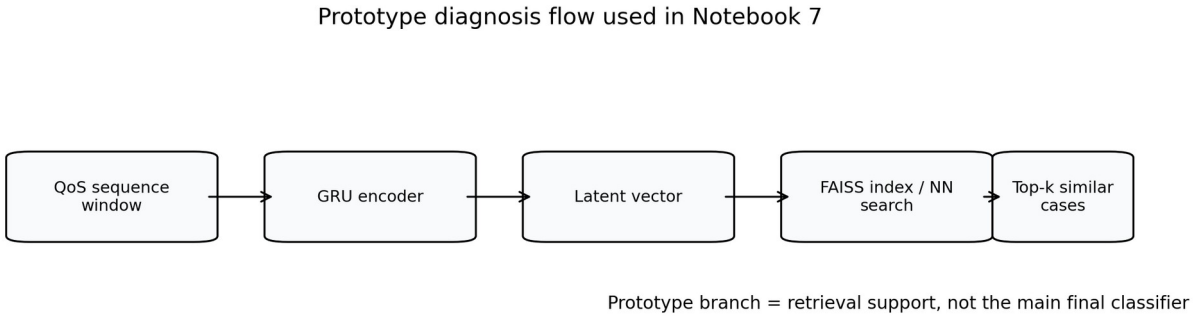



*Temporal comparison between LSTM and GRU autoencoders.*

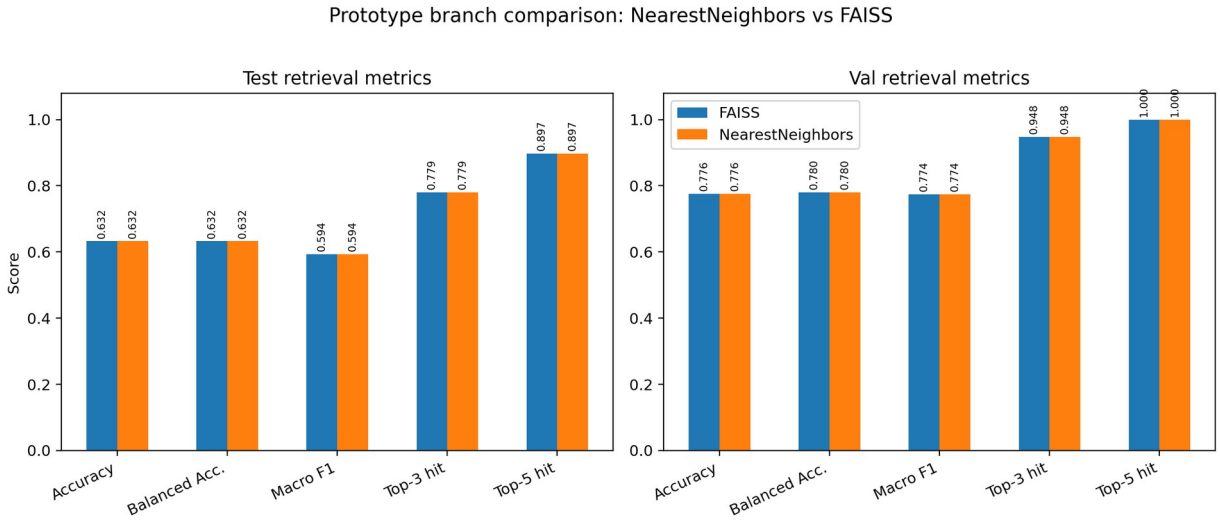
## 4.4 Prototype Methods

The prototype branch does not replace the classifier. Its role is to retrieve similar historical patterns in latent space. The GRU encoder transforms each sequence window into a latent vector. Prototype retrieval then compares a new latent vector with stored training vectors.

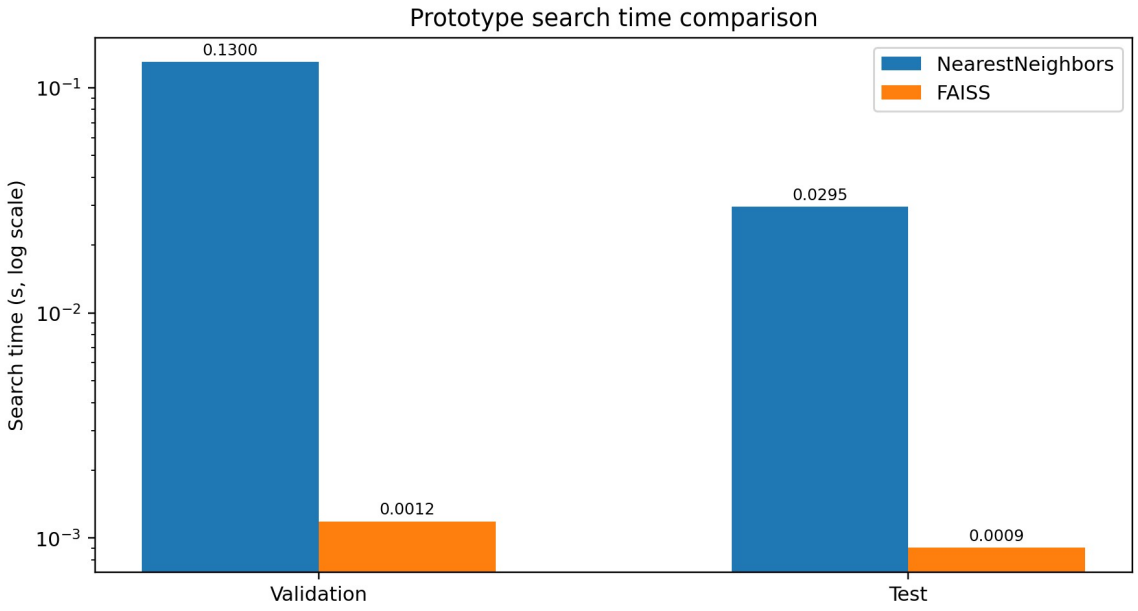
- NearestNeighbors performs exact nearest-neighbor search in the latent space. It is simple and exact, which makes it a clean baseline.
- FAISS is a vector-search backend designed for fast similarity retrieval. In this benchmark it used the same latent space, so quality can be compared directly against NearestNeighbors.
- In the Diagnostic Agent, this branch supports similar-case retrieval, candidate explanation, and operator-facing evidence rather than acting as the sole final classifier.




*How sequence windows become latent vectors and then prototype retrieval candidates.*




*Prototype-method comparison between NearestNeighbors and FAISS.*




*FAISS keeps similar quality but is much faster for search.*

## 5. Training and Evaluation Strategy

The training and evaluation design was chosen to stay faithful to the business use case of the Diagnostic Agent. Because this is QoS data with temporal structure, the benchmark avoided a random 80/20 split and instead used chronological train/validation/test splits with a purge gap. This reduces temporal leakage and gives a more realistic estimate of future performance.

| Branch | Metrics used | Why these metrics were chosen |
|---|---|---|
| Supervised tabular models | Accuracy, Balanced Accuracy, Macro F1, Weighted F1, Macro Precision, Macro Recall, ROC AUC, PR AUC | Accuracy is intuitive; Balanced Accuracy and Macro F1 prevent a single class from dominating interpretation; ROC AUC and PR AUC evaluate ranking quality without fixing one threshold. |
| Temporal autoencoders | Validation reconstruction loss, mean/median test reconstruction error, latent silhouette, training time | Reconstruction quality is the core objective of an autoencoder; mean and median test errors show stability; silhouette gives a rough indication of latent separability; training time matters for practical reuse. |
| Prototype methods | Accuracy, Balanced Accuracy, Macro F1, top-3 hit rate, top-5 hit rate, search time | Prototype retrieval is not only about top-1 class prediction: top-k hit rate tells whether the correct class appears among the retrieved candidates; search time matters operationally for online use. |

# 6. Results and Final Selection

## 6.1 Supervised branch results

| Model | Validation Macro F1 | Validation ROC AUC | Test Macro F1 | Test ROC AUC |
|---|---|---|---|---|
| Random Forest | 0.973 | 0.988 | 0.988 | 0.989 |
| XGBoost | 0.973 | 0.980 | 0.988 | 0.987 |

Interpretation: both supervised models were very strong after the patch, but Random Forest won because it matched XGBoost on the main classification metrics and had slightly stronger validation tie-break metrics.

## 6.2 Temporal branch results

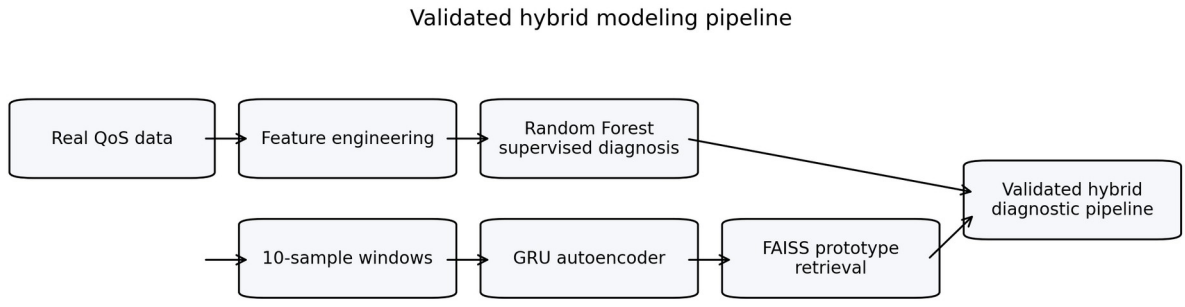
| Model | Best validation loss | Best epoch | Mean test reconstruction error | Latent silhouette |
|---|---|---|---|---|
| LSTM | 0.402 | 30 | 0.508 | 0.032 |
| GRU | 0.391 | 29 | 0.455 | 0.013 |

Interpretation: GRU reconstructed the benchmark windows better than LSTM. LSTM had a slightly better silhouette score, but both silhouette values were low, so reconstruction performance was the more reliable selection criterion.

## 6.3 Prototype branch results

| Method | Validation Macro F1 | Test Macro F1 | Test top-3 hit rate | Test top-5 hit rate | Test search time (s) |
|---|---|---|---|---|---|
| NearestNeighbors | 0.774 | 0.594 | 0.779 | 0.897 | 0.0295 |
| FAISS | 0.774 | 0.594 | 0.779 | 0.897 | 0.0009 |

Interpretation: prototype top-1 accuracy is moderate, but top-3 and top-5 hit rates are much stronger. This supports the intended role of the prototype branch as a similar-case retrieval mechanism rather than the main final classifier. FAISS won because it preserved quality while making retrieval far faster.




*The validated hybrid pipeline after model selection.*

# 7. Final Selected Models

| Branch | Selected winner | Reason for selection |
|---|---|---|
| Supervised tabular diagnosis | Random Forest | Best corrected baseline after |

| | | |
|---|---|---|
| | | removing risky features; strong val/test performance. |
| Temporal representation | GRU | Lower validation loss and lower mean test reconstruction error. |
| Prototype retrieval | FAISS | Same validation quality as NearestNeighbors but dramatically faster search time. |

## 8. Practical Presentation Narrative

A clear way to present the modeling validation is to explain it as a hybrid benchmark. The benchmark first validates a corrected supervised classifier branch on robust real-data labels. It then validates a temporal encoder that reconstructs QoS windows and builds a latent representation. Finally, it validates a prototype retrieval layer that can surface similar historical cases quickly. This narrative keeps the story operational, explainable, and aligned with the business role of the Diagnostic Agent.

- We trained only on real data and kept the benchmark honest by avoiding synthetic augmentation.
- We used chronological train/validation/test splits with purge gaps instead of random splitting.
- We kept the supervised target space restricted to the classes that were actually supported robustly by the current data.
- We selected one winner per branch so that the final validated pipeline is easy to explain and defend.

## 9. Conclusion

The final modeling validation produces a coherent and explainable hybrid outcome for the current benchmark. Random Forest is the best corrected supervised baseline for structured KPI diagnosis. GRU Autoencoder is the best temporal model for reconstructing 10-sample QoS windows. FAISS is the best prototype-search backend because it retains retrieval quality while making search much faster. Together, these components form a clear first validated version of the Diagnostic Agent modeling stack.

## Appendix A — Full List of Training Columns Used in the Corrected Benchmark

The corrected supervised notebook used 104 columns after removing 14 risky or shortcut features. The list below is the exact feature set used to build the corrected supervised benchmark.

| Column group 1 | Column group 2 | Column group 3 | Column group 4 |
|---|---|---|---|
| active_connections | cqi | latency_ms_delta_5 | queue_length_rollstd_3 |
| bandwidth_util_pct | cssr_proxy_pct | latency_ms_rollmean_3 | queue_length_rollstd_5 |
| bandwidth_util_pct_delta_3 | earfcn | latency_ms_rollmean_5 | radio_efficiency_score |
| bandwidth_util_pct_delta_5 | flag_high_util | latency_ms_rollstd_3 | radio_vs_transport_score |
| bandwidth_util_pct_rollmean_3 | flag_latency_crit | latency_ms_rollstd_5 | rsrp_dbm |
| bandwidth_util_pct_rollmean_5 | flag_latency_warn | latency_rolling_mean | rsrq_db |
| bandwidth_util_pct_rollstd_3 | flag_pl_crit | latency_rolling_std | rssi_dbm |
| bandwidth_util_pct_rollstd_5 | flag_pl_warn | latency_trend | signal_degradation_rate |
| bler_delta | flag_queue_high | latency_volatility | signal_health_score |
| bler_mcs_stress | handover_count | mcs | signal_quality_pct |
| bler_pct_model | handover_event | memory_pct | sinr_db |
| bler_pct_model_delta_3 | handover_instability_index | mos_estimate | tcp_retransmit_rate |
| bler_pct_model_delta_5 | ho_success_rate_pct | neighbor_count | throughput_loss_explainer |
| bler_pct_model_rollmean_3 | jitter_increasing | packet_loss_pct | throughput_mbps |
| bler_pct_model_rollmean_5 | jitter_ms | packet_loss_pct_delta_3 | throughput_mbps_delta_3 |
| bler_pct_model_rollstd_3 | jitter_ms_delta_3 | packet_loss_pct_delta_5 | throughput_mbps_delta_5 |
| bler_pct_model_rollstd_5 | jitter_ms_delta_5 | packet_loss_pct_rollmean_3 | throughput_mbps_rollmean_3 |
| bler_pressure_score | jitter_ms_rollmean_3 | packet_loss_pct_rollmean_5 | throughput_mbps_rollmean_5 |
| bler_proxy_pct | jitter_ms_rollmean_5 | packet_loss_pct_rollstd_3 | throughput_mbps_rollstd_3 |
| bler_sinr_gap | jitter_ms_rollstd_3 | packet_loss_pct_rollstd_5 | throughput_mbps_rollstd_5 |
| cellular_signal_score | jitter_ms_rollstd_5 | pci | throughput_rolling_mean |
| channel | jitter_rolling_mean | queue_length | throughput_rolling_std |
| channel_util_pct | jitter_rolling_std | queue_length_delta_3 | throughput_util_ratio |
| congestion_index | latency_jitter_ratio | queue_length_delta_5 | throughput_volatility |
| connected_stations | latency_ms | queue_length_rollmean_3 | transport_pressure_score |
| cpu_pct | latency_ms_delta_3 | queue_length_rollmean_5 | wifi_signal_score |